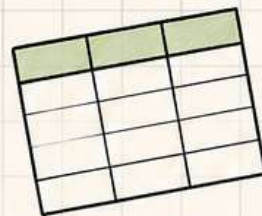


Simple.
Clear.
Useful.

SQL



Data in,
Insights out.

The Essential

SQL SYNTAX

A Visual Guide for Beginners, Made by a Beginner

- ✓ Sintaxis clara
- ✓ Ejemplos prácticos
- ✓ Fácil de consultar
- ✓ Lo esencial al alcance de tus manos



★
Aprendiendo SQL,
un query a la vez.



</>
Escribe hoy,
entiende mañana.

★
GuilleGiu

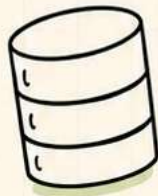
Designed & Summarized by GuilleGiu

id	name	age
1	Ana	25
2	Luis	30
3	Mia	22

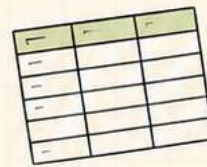


Este Cheat Sheet reúne la sintaxis de SQL que más se utiliza en el día a día para aprender, practicar y consultar rápidamente.





ÍNDICE



Encuentra rápido lo que necesitas.



1 PARTE 1 – CONSULTAS

- SELECT
- DISTINCT
- WHERE
- AND
- OR
- NOT
- LIKE
- IN
- BETWEEN
- ORDER BY
- LIMIT



2 PARTE 2 – MODIFICAR DATOS



- INSERT
- UPDATE
- DELETE

3 PARTE 3 – FUNCIONES



- COUNT
- SUM
- AVG
- MIN
- MAX
- GROUP BY
- HAVING

4 PARTE 4 – RELACIONES



- ALIASES
- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL JOIN
- CROSS JOIN
- SELF JOIN

5 PARTE 5 – CONSULTAS AVANZADAS



- UNION
- UNION ALL
- EXISTS
- ANY
- ALL
- INSERT INTO SELECT

6 PARTE 6 – UTILIDADES

- IS NULL
- IS NOT NULL
- IFNULL()
- COALESCE()

- CREATE
- ALTER
- DROP
- TRUNCATE



Sintaxis clara, ejemplos prácticos y lo esencial de SQL
para que aprendas más rápido.



GuilleGiu

Designed & Summarized by GuilleGiu

SQL QUICK SYNTAX

≡ PARTE 1 – CONSULTAS ≡

1. SELECT

→ Selecciona columnas específicas de una tabla.

```
SELECT column1, column2, column3  
FROM table_name;
```

2. SELECT *

→ Selecciona todas las columnas de una tabla.

```
SELECT *  
FROM table_name;
```

3. SELECT DISTINCT

→ Selecciona filas únicas (sin duplicados) en una columna o en un conjunto de columnas.

```
SELECT DISTINCT column1, column2  
FROM table_name;
```



SQL QUICK SYNTAX

≡ PARTE 1 – CONSULTAS ≡

2. WHERE, AND, OR, NOT

→ Filtra registros usando condiciones y operadores lógicos.

- WHERE

```
SELECT column1, column2  
FROM table_name  
WHERE condition;
```

- AND

```
SELECT *  
FROM table_name  
WHERE condition1 AND condition2;
```

- OR

```
SELECT *  
FROM table_name  
WHERE condition1 OR condition2;
```

- NOT

```
SELECT *  
FROM table_name  
WHERE NOT condition;
```



SQL QUICK SYNTAX

≡ PARTE 1 – CONSULTAS ≡

3. LIKE, IN, BETWEEN

→ Operadores para buscar patrones, valores dentro de una lista o rangos de valores.

- LIKE

```
SELECT column1, column2
FROM table_name
WHERE column LIKE 'pattern';
```

Notas:

Usa % para cualquier cantidad de caracteres y _ para un solo carácter.

- IN

```
SELECT column1, column2
FROM table_name
WHERE column IN (value1, value2, value3, ...);
```

Notas:

Verifica si un valor está dentro de una lista de valores.

- BETWEEN

```
SELECT column1, column2
FROM table_name
WHERE column BETWEEN value1 AND value2;
```

Notas:

Selecciona valores dentro de un rango (incluye los extremos).



≡ GuilleGiu Notes ≡

Designed & Summarized by GuilleGiu

SQL QUICK SYNTAX

≡ PARTE 1 – CONSULTAS ≡

4. ORDER BY, ASC, DESC, LIMIT

→ Ordena y limita los resultados de una consulta.

- ORDER BY (ASC)

```
SELECT column1, column2  
FROM table_name  
ORDER BY column1 ASC;
```

Notas:

ASC = Ascendente (de menor a mayor).

- ORDER BY (DESC)

```
SELECT column1, column2  
FROM table_name  
ORDER BY column1 DESC;
```

Notas:

DESC = Descendente (de mayor a menor).

- ORDER BY múltiples columnas

```
SELECT column1, column2, column3  
FROM table_name  
ORDER BY column1 ASC, column2 DESC;
```

Notas:

Se ordena primero por column1 y luego por column2.

- LIMIT

```
SELECT column1, column2  
FROM table_name  
LIMIT 10;
```

Notas:

Limita el número de filas devueltas.



SQL QUICK SYNTAX

≡ PARTE 2 – MODIFICAR DATOS ≡

5. INSERT

→ Inserta nuevos registros en una tabla.

Sintaxis 1: Especificando columnas y valores

- Recomendado. Especifica las columnas y los valores a insertar.

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

Ejemplo:

```
INSERT INTO Customers (CustomerID, CustomerName, City)  
VALUES (101, 'Ana López', 'Madrid');
```

Sintaxis 2: Sin especificar columnas (todas las columnas)

- Debes respetar el orden de las columnas definido en la tabla.

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

Ejemplo:

```
INSERT INTO Customers  
VALUES (102, 'Carlos Ruiz', 'Barcelona', 'Spain');
```

Insertar múltiples filas

- Inserta varios registros en una sola sentencia.

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...),  
       (value1, value2, value3, ...),  
       (value1, value2, value3, ...);
```

Ejemplo:

```
INSERT INTO Employees (ID, Name, Department)  
VALUES (1, 'Lucía', 'HR'),  
       (2, 'Pedro', 'IT'),  
       (3, 'María', 'Finance');
```



≡ PARTE 2 – MODIFICAR DATOS ≡

6. UPDATE

→ Modifica registros existentes en una tabla.

Sintaxis 1: Actualizar una o más columnas

```
UPDATE Customers
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Ejemplo:

```
UPDATE Customers
SET City = 'Madrid'
WHERE CustomerID = 101;
```

Notas:

- La cláusula WHERE es muy importante para evitar actualizar todos los registros.

Sintaxis 2: Actualizar todas las columnas

```
UPDATE Customers
SET column1 = value1, column2 = value2, column3 = value3, ...
WHERE condition;
```

Ejemplo:

```
UPDATE Customers
SET CustomerName = 'Juan Pérez', City = 'Barcelona', Country = 'Spain'
WHERE CustomerID = 102;
```

Sintaxis 3: Usar condiciones para actualizar

```
UPDATE Customers
SET City = 'Valencia'
WHERE Country = 'Spain' AND City = 'Madrid';
```

Ejemplo:

```
UPDATE Customers
SET Country = 'Argentina'
WHERE City = 'Buenos Aires' AND CustomerID > 100;
```



≡ PARTE 2 – MODIFICAR DATOS ≡

7. DELETE

→ Elimina registros de una tabla.

Sintaxis 1: Eliminar registros específicos

```
DELETE FROM table_name  
WHERE condition;
```

Ejemplo:

```
DELETE FROM Customers  
WHERE CustomerID = 101;
```

Notas:

- Se eliminarán todos los registros que cumplan la condición.
- Si omites **WHERE**, se eliminarán todos los registros de la tabla.

Sintaxis 2: Eliminar todos los registros (**usar con precaución!**)

```
DELETE FROM table_name;
```

Ejemplo:

```
DELETE FROM Customers;
```

Notas:

- Elimina todos los registros de la tabla, pero la estructura de la tabla se mantiene.

Sintaxis 3: Eliminar usando subconsulta

```
DELETE FROM table_name  
WHERE column IN (SELECT column FROM another_table WHERE condition);
```

Ejemplo:

```
DELETE FROM Customers  
WHERE CustomerID IN (  
    SELECT CustomerID FROM Orders WHERE OrderDate < '2024-01-01'  
);
```

Notas:

- Elimina clientes que tengan órdenes anteriores a la fecha indicada.



SQL QUICK SYNTAX

≡ PARTE 3 – FUNCIONES ≡

→ Las funciones de agregación permiten obtener un resumen de los datos.

- COUNT() : Cuenta registros.
- SUM() : Suma valores numéricos.
- AVG() : Promedio de valores numéricos.
- MIN() : Valor mínimo.
- MAX() : Valor máximo.

• COUNT() – Cuenta registros

```
SELECT COUNT(column1) AS total_registros  
FROM Customers;
```

Ejemplo:

- Cuenta el número total de clientes.

• SUM() – Suma valores numéricos

```
SELECT SUM(column1) AS total_ventas  
FROM Customers;
```

Ejemplo:

- Suma los valores de la columna City (longitud del nombre de la ciudad).

• AVG() – Promedio de valores numéricos

```
SELECT AVG(column1) AS promedio_longitud  
FROM Customers;
```

Ejemplo:

- Calcula el promedio de la longitud de los nombres de las ciudades.

• MIN() – Valor mínimo

```
SELECT MIN(column1) AS ciudad_mas_corta  
FROM Customers;
```

Ejemplo:

- Encuentra la ciudad con el nombre más corto.

• MAX() – Valor máximo

```
SELECT MAX(column1) AS ciudad_mas_larga  
FROM Customers;
```

Ejemplo:

- Encuentra la ciudad con el nombre más largo.



SQL QUICK SYNTAX

PARTE 3 - FUNCIONES

9. GROUP BY + HAVING

→ Agrupa filas que tienen el mismo valor y permite aplicar condiciones a los grupos.

● GROUP BY - Agrupar resultados

```
SELECT column_name, AGG_FUNC(column_name)
FROM table_name
GROUP BY column_name;
```

GROUP BY agrupa las filas por los valores de la columna especificada.

Ejemplo:

- Cuenta cuántos clientes hay en cada país.

```
SELECT Country, COUNT(CustomerID) AS TotalClientes
FROM Customers
GROUP BY Country;
```

Country	TotalClientes
Argentina	3
Spain	2
USA	2
Germany	1
...	...

● HAVING - Filtrar grupos

```
SELECT column_name, AGG_FUNC(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

HAVING se usa después de GROUP BY para filtrar grupos (no filas individuales).

Ejemplo:

- Mostrar solo los países con más de 2 clientes.

```
SELECT Country, COUNT(CustomerID) AS TotalClientes
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 2;
```

Country	TotalClientes
Argentina	3
...	...

● GROUP BY + HAVING - Ejemplo completo

- Promedio de clientes por ciudad, mostrando solo ciudades con promedio > 1.

```
SELECT City, AVG(CustomerID) AS PromedioID,
COUNT(CustomerID) AS TotalClientes
FROM Customers
GROUP BY City
HAVING AVG(CustomerID) > 1
ORDER BY PromedioID DESC;
```

City	PromedioID	TotalClientes
Madrid	102.5	2
Barcelona	76.0	3
Buenos Aires	60.0	2
...	...	...

★ RECUERDA

- WHERE filtra filas antes de agrupar.
- GROUP BY crea los grupos.
- HAVING filtra los grupos después de agrupar.
- Las funciones de agregación (COUNT, SUM, AVG, MIN, MAX) se usan con GROUP BY.



SQL QUICK SYNTAX

PARTE 4 - RELACIONES

9. ALIASES

- Alias (apodos) para tablas o columnas.
Hacen que las consultas sean más cortas y fáciles de leer.

● ALIAS PARA TABLAS

Sintaxis:

```
SELECT column_name(s)
FROM table_name AS alias
WHERE condition;
```



AS es opcional.
También podés omitirlo:
table_name alias
(sin AS)

Ejemplo:

```
SELECT c.CustomerName, c.City, c.Country
FROM Customers AS c
WHERE c.Country = 'Argentina';
```

→ **c** = alias de Customers

● ALIAS PARA COLUMNAS

Sintaxis:

```
SELECT column_name AS alias
FROM table_name;
```



Muy útil para renombrar columnas en el resultado.

Ejemplo:

```
SELECT CustomerName AS Cliente,
       City AS Ciudad,
       Country AS País
FROM Customers;
```

Resultado:

Cliente	Ciudad	País
Juan Pérez	Madrid	Spain
María Gómez	Barcelona	Spain
Carlos López	Buenos Aires	Argentina
...	...	...

● ALIAS PARA TABLAS Y COLUMNAS

Ejemplo:

```
SELECT c.CustomerName AS Cliente,
       c.City AS Ciudad,
       c.Country AS País
FROM Customers AS c
WHERE c.City = 'Madrid';
```

Resultado:

Cliente	Ciudad	País
Juan Pérez	Madrid	Spain
Ana Torres	Madrid	Spain
...	...	...



RECORDÁ

- Los alias mejoran la legibilidad de tus consultas.
- Usá alias cortos y descriptivos (ej: **c** para Customers, **o** para Orders).
- Los alias no existen fuera de la consulta (solo duran en esa consulta).

Alias = claridad
en menos
tiempo.



SQL QUICK SYNTAX

PARTE 4 - RELACIONES

10. INNER JOIN, LEFT JOIN, RIGHT JOIN Y FULL JOIN

→ Permiten combinar filas de dos tablas relacionadas según una condición.

Customers (c)		
CustomerID	CustomerName	City
1	Juan Pérez	Madrid
2	María Gómez	Barcelona
3	Carlos López	Buenos Aires
4	Ana Torres	Madrid

Orders (o)			
OrderID	CustomerID	OrderDate	Amount
101	1	2024-01-10	150.00
102	1	2024-02-15	200.00
103	2	2024-03-05	120.00
104	4	2024-03-18	80.00



Relación:
Customers.CustomerID
= Orders.CustomerID
(un cliente puede tener 0, 1 o muchos pedidos)

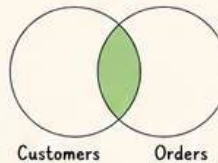
● INNER JOIN

Devuelve solo las filas que coinciden en ambas tablas.

Sintaxis:

```
SELECT c.CustomerName, o.OrderID, o.OrderDate, o.Amount
FROM Customers c
INNER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Coinciden en
ambas



Resultado:

CustomerName	OrderID	OrderDate	Amount
Juan Pérez	101	2024-01-10	150.00
Juan Pérez	102	2024-02-15	200.00
María Gómez	103	2024-03-05	120.00
Ana Torres	104	2024-03-18	80.00

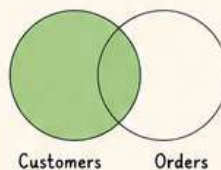
● LEFT JOIN

Devuelve todas las filas de la tabla izquierda (Customers) y las coincidentes de la derecha (Orders).
Si no hay coincidencia, muestra NULL en Orders.

Sintaxis:

```
SELECT c.CustomerName, o.OrderID, o.OrderDate, o.Amount
FROM Customers c
LEFT JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Todas las filas
de la izquierda



Resultado:

CustomerName	OrderID	OrderDate	Amount
Juan Pérez	101	2024-01-10	150.00
Juan Pérez	102	2024-02-15	200.00
María Gómez	103	2024-03-05	120.00
Ana Torres	104	2024-03-18	80.00
Carlos López	NULL	NULL	NULL

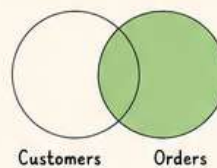
● RIGHT JOIN

Devuelve todas las filas de la tabla derecha (Orders) y las coincidentes de la izquierda (Customers).
Si no hay coincidencia, muestra NULL en Customers.

Sintaxis:

```
SELECT c.CustomerName, o.OrderID, o.OrderDate, o.Amount
FROM Customers c
RIGHT JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Todas las filas
de la derecha



Resultado:

CustomerName	OrderID	OrderDate	Amount
Juan Pérez	101	2024-01-10	150.00
Juan Pérez	102	2024-02-15	200.00
María Gómez	103	2024-03-05	120.00
Ana Torres	104	2024-03-18	80.00

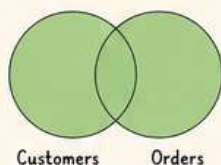
● FULL JOIN (FULL OUTER JOIN)

Devuelve todas las filas de ambas tablas.
Si no hay coincidencia, muestra NULL en la otra tabla.

Sintaxis:

```
SELECT c.CustomerName, o.OrderID, o.OrderDate, o.Amount
FROM Customers c
FULL OUTER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

Todas las filas
de ambas tablas



Resultado:

CustomerName	OrderID	OrderDate	Amount
Juan Pérez	101	2024-01-10	150.00
Juan Pérez	102	2024-02-15	200.00
María Gómez	103	2024-03-05	120.00
Ana Torres	104	2024-03-18	80.00
Carlos López	NULL	NULL	NULL

📌 RECUERDA

- INNER JOIN = solo coincidencias.
- LEFT JOIN = todas de la izquierda + coincidencias.
- RIGHT JOIN = todas de la derecha + coincidencias.
- FULL JOIN = todas de ambas tablas.

Usá alias (c, o)
para que tus consultas
sean más cortas y
fáciles de entender.

Orden lógico de ejecución (resumen)



≡ PARTE 4 - RELACIONES ≡

11. CROSS JOIN + SELF JOIN

● CROSS JOIN

→ Devuelve el producto cartesiano entre dos tablas: combina cada fila de la primera con todas las filas de la segunda.

Sintaxis:

```
SELECT column_list
FROM table1
CROSS JOIN table2;
```

Ejemplo:

```
SELECT c.CustomerName, co.Country
FROM Customers c
CROSS JOIN Countries co;
```

Customers (c)	
CustomerID	CustomerName
1	Juan Pérez
2	María Gómez
3	Carlos López

Countries (co)	
CountryID	Country
1	Argentina
2	Spain
3	USA



¿Qué hace?

Cada cliente se combina con cada país (3 x 3 = 9 filas).

Resultado:

CustomerName	Country
Juan Pérez	Argentina
Juan Pérez	Spain
Juan Pérez	USA
María Gómez	Argentina
María Gómez	Spain
María Gómez	USA
Carlos López	Argentina
Carlos López	Spain
Carlos López	USA

☆ ÚTIL PARA:

- Generar todas las combinaciones posibles entre dos conjuntos.
- Reportes, catálogos o análisis.

● SELF JOIN

→ Es un JOIN de una tabla consigo misma. Muy útil para datos jerárquicos.

Sintaxis:

```
SELECT a.Column_list, b.column_list
FROM table_name a
JOIN table_name b ON condition;
```

Ejemplo:

```
SELECT a.CustomerName AS Cliente,
       b.CustomerName AS Mismo_City,
       a.City
FROM Customers a
JOIN Customers b ON a.City = b.City
AND a.CustomerID <> b.CustomerID
ORDER BY a.City, a.CustomerName;
```

Usamos alias (a, b) para tratar cada copia de la tabla de forma distinta.



¿Qué hace?

Encuentra clientes que viven en la misma ciudad, excluyendo al mismo cliente (a.CustomerID <> b.CustomerID).

Customers (c)		
CustomerID	CustomerName	City
1	Juan Pérez	Madrid
2	María Gómez	Barcelona
3	Carlos López	Buenos Aires
4	Ana Torres	Madrid

Resultado:

Cliente	Mismo_City	City
Juan Pérez	Ana Torres	Madrid
Ana Torres	Juan Pérez	Madrid
María Gómez	Carlos López	Barcelona
Carlos López	María Gómez	Barcelona

☆ ÚTIL PARA:

- Comparar filas de la misma tabla.
- Relaciones jerárquicas (ej: empleados y jefes).
- Encontrar duplicados o coincidencias.

Ejemplo jerárquico (empleados y jefes)

```
SELECT e.CustomerName AS Empleado,
       j.CustomerName AS Jefe,
       e.City
FROM Customers e
JOIN Customers j ON e.City = j.City
AND e.CustomerID <> j.CustomerID;
```

⚠ A tener en cuenta ⚠

Los SELF JOIN pueden generar muchas filas. Siempre filtrá bien la condición.

SQL QUICK SYNTAX

PARTE 5 - CONSULTAS AVANZADAS

12. UNION

→ Combina el resultado de dos o más consultas en una sola lista, eliminando los duplicados.

Sintaxis:

```
SELECT columna1, columna2 FROM tabla1
UNION
SELECT columna1, columna2 FROM tabla2;
```



UNION elimina filas duplicadas. Los resultados deben tener el mismo número de columnas y tipos de datos compatibles.

Ejemplo: obtener clientes de Argentina y España (sin duplicados)

```
SELECT CustomerName, Country FROM Customers
WHERE Country = 'Argentina'
UNION
SELECT CustomerName, Country FROM Customers
WHERE Country = 'Spain';
```

Resultado:

CustomerName	Country
Juan Pérez	Argentina
María Gómez	Spain
Carlos López	Argentina
Ana Torres	Spain
...	...

13. UNION ALL

→ Combina el resultado de dos o más consultas en una sola lista, pero mantiene los duplicados.

Sintaxis:

```
SELECT columna1, columna2 FROM tabla1
UNION ALL
SELECT columna1, columna2 FROM tabla2;
```



UNION ALL es más rápido porque no verifica ni elimina duplicados.

Ejemplo: clientes de Argentina y España (mantiene duplicados)

```
SELECT CustomerName, Country FROM Customers
WHERE Country = 'Argentina'
UNION ALL
SELECT CustomerName, Country FROM Customers
WHERE Country = 'Spain';
```

Resultado:

CustomerName	Country
Juan Pérez	Argentina
María Gómez	Spain
Carlos López	Argentina
Ana Torres	Spain
Juan Pérez	Argentina
Carlos López	Argentina
...	...

★ RECUERDA

- UNION elimina duplicados.
- UNION ALL mantiene duplicados.
- Las consultas deben tener el mismo número de columnas y tipos compatibles.
- Se usa comúnmente para combinar datos de tablas relacionadas.

Piensa en UNION como juntar listas sin repeticiones y UNION ALL como juntar listas tal cual están.



SQL QUICK SYNTAX

PARTE 5 - CONSULTAS AVANZADAS

13. EXISTS, ANY, ALL



IDEA GENERAL

Sirven para comparar valores o verificar existencia de filas en otra consulta (subconsulta).

Customers (c)			
CustomerID	CustomerName	City	Country
1	Juan Pérez	Madrid	Spain
2	María Gómez	Barcelona	Spain
3	Carlos López	Buenos Aires	Argentina
4	Ana Torres	Madrid	Spain
5	Luis Martín	Santiago	Chile
6	Pedro Ruiz	Lima	Peru

Orders (o)			
OrderID	CustomerID	OrderDate	Amount
101	1	2024-01-10	150.00
102	1	2024-02-15	200.00
103	2	2024-01-20	120.00
104	3	2024-03-05	180.00
105	3	2024-04-12	250.00
106	4	2024-02-28	80.00

A. EXISTS

→ Devuelve TRUE si la subconsulta retorna UNA O MÁS filas.
Se usa para comprobar existencia de relaciones.

Sintaxis:

```
SELECT columnas
FROM tabla_exterior e
WHERE EXISTS (
  SELECT 1
  FROM tabla_interior i
  WHERE i.columna = e.columna
);
```

Ejemplo:

```
SELECT c.CustomerName
FROM Customers c
WHERE EXISTS (
  SELECT 1
  FROM Orders o
  WHERE o.CustomerID = c.CustomerID
);
```

Resultado:

CustomerName
Juan Pérez
María Gómez
Carlos López
Ana Torres

★ Solo clientes que TIENEN al menos un pedido.

B. ANY (SOME)

→ Devuelve TRUE si la comparación es válida para AL MENOS UN valor de la subconsulta.

Sintaxis:

```
SELECT columnas
FROM tabla_exterior
WHERE columna OPERATOR ANY (
  SELECT columna
  FROM tabla_interior
);
```

Ejemplo:

```
SELECT c.CustomerName, c.City
FROM Customers c
WHERE c.CustomerID = ANY (
  SELECT o.CustomerID
  FROM Orders o
  WHERE o.Amount > 150
);
```

Resultado:

CustomerName	City
Juan Pérez	Madrid
María Gómez	Barcelona
Carlos López	Buenos Aires

★ Clientes que hicieron al menos un pedido mayor a 150.

C. ALL

→ Devuelve TRUE si la comparación es válida para TODOS los valores de la subconsulta.

Sintaxis:

```
SELECT columnas
FROM tabla_exterior
WHERE columna OPERATOR ALL (
  SELECT columna
  FROM tabla_interior
);
```

Ejemplo:

```
SELECT c.CustomerName, c.City
FROM Customers c
WHERE c.CustomerID > ALL (
  SELECT o.CustomerID
  FROM Orders o
);
```

Resultado:

CustomerName	City
Luis Martín	Santiago
Pedro Ruiz	Lima

★ Clientes cuyo ID es mayor que TODOS los CustomerID de Orders.



COMPARACIONES MÁS USADAS

- = ANY → igual a al menos uno (equivalente a IN)
- > ANY → mayor que al menos uno
- < ANY → menor que al menos uno
- = ALL → igual a todos
- > ALL → mayor que todos
- < ALL → menor que todos

EJEMPLOS RÁPIDOS

- ID = ANY (subconsulta)
↔ ID IN (subconsulta)
- EXISTS es ideal para relaciones (más eficiente).
- ANY y ALL comparan valores.



CUÁNDO USAR QUÉ

- EXISTS → ¿Existe al menos una fila relacionada?
- ANY → ¿Cumple con al menos un valor de la subconsulta?
- ALL → ¿Cumple con todos los valores de la subconsulta?

PARTE 5 - CONSULTAS AVANZADAS

14. INSERT INTO SELECT

→ Permite insertar resultados de una consulta (SELECT) en otra tabla.

SINTAXIS BÁSICA:

```
INSERT INTO tabla_destino (columna1, columna2, ...)
SELECT columna1, columna2, ...
FROM tabla_origen
WHERE condicion (opcional);
```



Útil para copiar datos, migrar información o crear respaldos de tablas.

IMPORTANTE

El número y orden de columnas en el SELECT debe coincidir con las columnas en el INSERT INTO.

● EJEMPLO 1: COPIAR TODOS LOS CLIENTES A OTRA TABLA

→ Creamos una tabla de respaldo y copiamos todos los datos de Customers.

Tablas:

Customers (origen)			
CustomerID	CustomerName	City	Country
1	Juan Pérez	Madrid	Spain
2	Maria Gómez	Barcelona	Spain
3	Carlos López	Buenos Aires	Argentina
4	Ana Torres	Madrid	Spain
...	...	...	...

COPIAMOS
→
TODOS LOS
DATOS

Customers_Backup (destino)			
CustomerID	CustomerName	City	Country
1	Juan Pérez	Madrid	Spain
2	Maria Gómez	Barcelona	Spain
3	Carlos López	Buenos Aires	Argentina
4	Ana Torres	Madrid	Spain
...	...	...	...

Consulta:

```
INSERT INTO Customers_Backup
SELECT *
FROM Customers;
```

★ La tabla destino debe existir antes de ejecutar la consulta.

● EJEMPLO 2: COPIAR CLIENTES DE UN PAÍS ESPECÍFICO

→ Copiamos solo los clientes de 'Spain' a la tabla Customers_Spain.

Tablas:

Customers (origen)			
CustomerID	CustomerName	City	Country
1	Juan Pérez	Madrid	Spain
2	Maria Gómez	Barcelona	Spain
3	Carlos López	Buenos Aires	Argentina
4	Ana Torres	Madrid	Spain
5	Luis Martín	Santiago	Chile
6	Pedro Ruiz	Lima	Peru
...	...	...	...

COPIAMOS
→
SOLO LOS
DE SPAIN

Customers_Spain (destino)			
CustomerID	CustomerName	City	Country
1	Juan Pérez	Madrid	Spain
2	Maria Gómez	Barcelona	Spain
4	Ana Torres	Madrid	Spain
...	...	...	...

Consulta:

```
INSERT INTO Customers_Spain (CustomerID, CustomerName, City, Country)
SELECT CustomerID, CustomerName, City, Country
FROM Customers
WHERE Country = 'Spain';
```

★ Elegimos las columnas que queremos copiar. Puedes usar * si quieres todas.

● EJEMPLO 3: INSERTAR CON TRANSFORMACIONES

→ Insertamos clientes, pero cambiando el valor de la columna Country a mayúsculas.

Tablas:

Customers (origen)			
CustomerID	CustomerName	City	Country
1	Juan Pérez	Madrid	Spain
2	Maria Gómez	Barcelona	Spain
3	Carlos López	Buenos Aires	Argentina
4	Ana Torres	Madrid	Spain
...	...	...	...

PAIS EN
→
MAYÚSCULAS

Customers_Upper (destino)			
CustomerID	CustomerName	City	Country
1	Juan Pérez	Madrid	SPAIN
2	Maria Gómez	Barcelona	SPAIN
3	Carlos López	Buenos Aires	ARGENTINA
4	Ana Torres	Madrid	SPAIN
...	...	...	...

Consulta:

```
INSERT INTO Customers_Upper (CustomerID, CustomerName, City, Country)
SELECT CustomerID, CustomerName, City, UPPER(Country)
FROM Customers;
```

★ Puedes aplicar funciones o expresiones en el SELECT antes de insertar.



REGLAS IMPORTANTES

- La tabla destino debe existir.
- Coincidir número y orden de columnas.
- Tipos de datos compatibles.
- Los datos contemplan las restricciones (NOT NULL, UNIQUE, etc).

CASOS DE USO

- ✓ Copias de seguridad
- ✓ Migración de datos
- ✓ Reportes y análisis
- ✓ Poblado de tablas
- ✓ Limpieza y transformación de datos



ERRORES COMUNES

- ✗ Columnas en distinto orden o cantidad.
- ✗ Tipos de datos incompatibles.
- ✗ Olvidar el WHERE y copiar más datos de los necesarios.

SQL QUICK SYNTAX

PARTE 6 - UTILIDADES

15. NULL (IS NULL - IS NOT NULL - IFNULL/COALESCE)



NULL representa un valor desconocido, no aplicado o inexistente.
NULL **NO** es igual a 0 ni a una cadena vacía "".

Tabla de ejemplo: Customers

CustomerID	CustomerName	City	Country	Phone
1	Juan Pérez	Madrid	Spain	600123456
2	María Gómez	Barcelona	Spain	NULL
3	Carlos López	Buenos Aires	Argentina	1145678901
4	Ana Torres	NULL	Spain	NULL
5	Luis Martín	Santiago	Chile	998877665
6	Pedro Ruiz	Lima	Peru	NULL

A. IS NULL

→ Devuelve las filas donde la columna es NULL.

Sintaxis:

```
SELECT columna(s)
FROM tabla
WHERE columna IS NULL;
```

Ejemplo:

```
SELECT CustomerName, Phone
FROM Customers
WHERE Phone IS NULL;
```

Resultado:

CustomerName	Phone
María Gómez	NULL
Ana Torres	NULL
Pedro Ruiz	NULL

B. IS NOT NULL

→ Devuelve las filas donde la columna **NO** es NULL.

Sintaxis:

```
SELECT columna(s)
FROM tabla
WHERE columna IS NOT NULL;
```

Ejemplo:

```
SELECT CustomerName, Phone
FROM Customers
WHERE Phone IS NOT NULL;
```

Resultado:

CustomerName	Phone
Juan Pérez	600123456
Carlos López	1145678901
Luis Martín	998877665

C. IFNULL() / COALESCE()

→ Permiten reemplazar valores NULL por un valor alternativo.

IFNULL(expr, valor)
(MySQL)

Si expr es NULL, devuelve valor.
Si no, devuelve expr.

COALESCE(expr1, expr2, ...)
(Estándar SQL)

Devuelve el primer valor que
NO es NULL de la lista.

¿CUÁNDO USAR?

- ☒ Mostrar valores por defecto.
- ☒ Evitar NULL en reportes.
- ☒ Combinar datos de varias columnas posibles.

Ejemplo 1: IFNULL()

```
SELECT CustomerName,
IFNULL(Phone, 'Sin teléfono') AS Phone
FROM Customers;
```

Resultado:

CustomerName	Phone
Juan Pérez	600123456
María Gómez	Sin teléfono
Carlos López	1145678901
Ana Torres	Sin teléfono
Luis Martín	998877665
Pedro Ruiz	Sin teléfono

Ejemplo 2: COALESCE() con varias columnas

```
SELECT CustomerName,
COALESCE(Phone, 'Sin teléfono', 'N/A') AS Contacto
FROM Customers;
```

Resultado:

CustomerName	Contacto
Juan Pérez	600123456
María Gómez	Sin teléfono
Carlos López	1145678901
Ana Torres	Sin teléfono
Luis Martín	998877665
Pedro Ruiz	Sin teléfono



NOTA IMPORTANTE

- NULL no se puede comparar con =, <, >, etc.
- Siempre usa IS NULL o IS NOT NULL.
- En expresiones, NULL produce NULL (salvo funciones como IFNULL/COALESCE).



ERRORES COMUNES

- ✗ WHERE columna = NULL (Siempre devuelve 0 filas).
- ✗ Usar = para buscar NULL.
- ✗ Olvidar que NULL ≠ 0 ni "".
- ✗ No considerar NULL en JOINS y agregaciones.



RESUMEN RÁPIDO

Concepto	Para qué sirve	Ejemplo
IS NULL	Encuentra valores NULL.	WHERE Phone IS NULL
IS NOT NULL	Encuentra valores que NO son NULL.	WHERE Phone IS NOT NULL
IFNULL(expr, valor)	Reemplaza NULL por un valor (MySQL).	IFNULL(Phone, 'Sin teléfono')
COALESCE(expr1, expr2, ...)	Devuelve el primer valor NO NULL.	COALESCE(Phone, AltPhone, 'N/A')



TIP EXTRA

En funciones agregadas:

COUNT(*) cuenta todas las filas (incluye NULLs).
COUNT(columna) cuenta solo las NO NULL.

PARTE 6 - UTILIDADES

16. CREATE - ALTER - DROP - TRUNCATE



Estos comandos se usan para crear, modificar y eliminar tablas o sus datos.
¡Úsalos con cuidado!

Tabla de ejemplo: Customers

CustomerID	CustomerName	City	Country	Phone
1	Juan Pérez	Madrid	Spain	600123456
2	María Gómez	Barcelona	Spain	NULL
3	Carlos López	Buenos Aires	Argentina	1145678901
4	Ana Torres	NULL	Spain	NULL

1 CREATE - Crear una tabla

→ Crea una nueva tabla con columnas y tipos de datos.

SINTAXIS:

```
CREATE TABLE nombre_tabla (
  columna1 tipo_dato [restricciones],
  columna2 tipo_dato [restricciones],
  ...
);
```

EJEMPLO:

```
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(100) NOT NULL,
  City VARCHAR(100),
  Country VARCHAR(100),
  Phone VARCHAR(20)
);
```

★ RECUERDA

- PRIMARY KEY → identifica cada fila de forma única.
- NOT NULL → no permite valores NULL.
- VARCHAR(n) → texto de hasta n caracteres.

2 ALTER - Modificar una tabla

→ Permite agregar, modificar o eliminar columnas.

SINTAXIS MÁS USADAS:

```
ALTER TABLE tabla ADD columna tipo; -- Agregar columna
ALTER TABLE tabla MODIFY columna tipo; -- Modificar tipo
ALTER TABLE tabla CHANGE old new tipo; -- Cambiar nombre y tipo
ALTER TABLE tabla DROP COLUMN columna; -- Eliminar columna
```

✎ NOTA

Algunos motores (como MySQL) usan MODIFY/CHANGE, otros (como SQL Server o PostgreSQL) usan ALTER COLUMN.

EJEMPLOS:

```
-- Agregar columna Email
ALTER TABLE Customers ADD Email VARCHAR(100);

-- Modificar tipo de Phone (más largo)
ALTER TABLE Customers MODIFY Phone VARCHAR(30);

-- Cambiar nombre de columna City a Location
ALTER TABLE Customers CHANGE City Location VARCHAR(100);

-- Eliminar columna Email
ALTER TABLE Customers DROP COLUMN Email;
```

3 DROP - Eliminar una tabla

→ Elimina la tabla completa junto con su estructura y datos.

SINTAXIS:

```
DROP TABLE nombre_tabla;
```

EJEMPLO:

```
DROP TABLE Customers;
```

⚠ ¡ATENCIÓN!

DROP elimina la tabla PARA SIEMPRE.
No se puede deshacer.

4 TRUNCATE - Vaciar una tabla

→ Elimina todos los datos de la tabla, pero mantiene su estructura.

SINTAXIS:

```
TRUNCATE TABLE nombre_tabla;
```

EJEMPLO:

```
TRUNCATE TABLE Customers;
```

✓ TRUNCATE vs DELETE

TRUNCATE	DELETE
Elimina todos los datos	Elimina filas (puedes usar WHERE)
Más rápido	Más lento
No activa triggers	Sí activa triggers
No se puede filtrar	Se puede filtrar con WHERE
No genera registros individuales en el log	Sí genera registros en el log

✓ RESUMEN RÁPIDO

Comando	¿Qué hace?	¿Afecta estructura?	¿Afecta datos?
CREATE	Crea una tabla nueva	SÍ	No aplica
ALTER	Modifica la estructura	SÍ	Puede (según el cambio)
DROP	Elimina la tabla completa	SÍ	SÍ (todo)
TRUNCATE	Elimina todos los datos	NO	SÍ (todo)

★ BUENAS PRÁCTICAS

- Siempre respalda antes de hacer cambios importantes.
- Prueba los cambios en un entorno de desarrollo.
- Escribe solo lo necesario y verifica antes de ejecutar.
- Los errores en producción pueden costar caro.

